Azure Multi-Service Cloud Application Report Documentation

Submission Screenshots Consolidated from Uploaded Archive

May 7, 2026

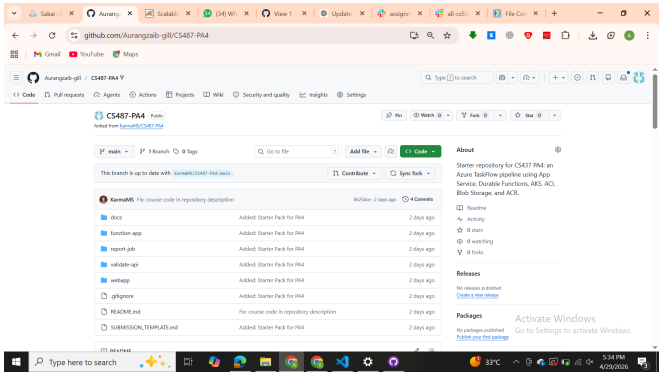
1. Overview

This document consolidates the uploaded evidence for the *Azure Multi-Service Cloud Application* assignment. It organizes the screenshots by task, adds short captions aligned with the assignment manual, and includes the required architecture discussion and system diagram in report form. The evidence shows an end-to-end TaskFlow deployment built from Azure App Service, Azure Container Registry (ACR), Azure Functions with Durable orchestration, Azure Kubernetes Service (AKS), Azure Container Instances (ACI), and Azure Blob Storage.

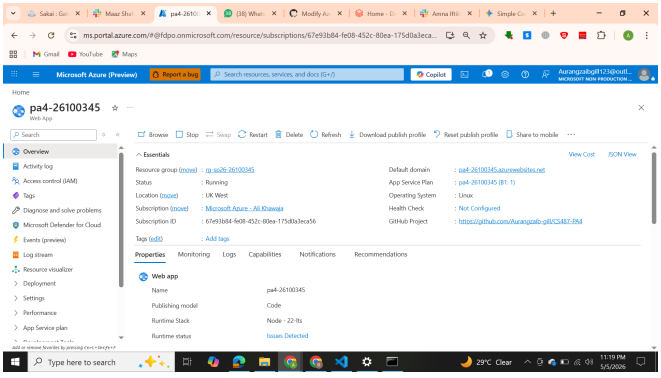
2. Task Evidence by Manual Section

2.1 Task 1: Web App Deployment from GitHub

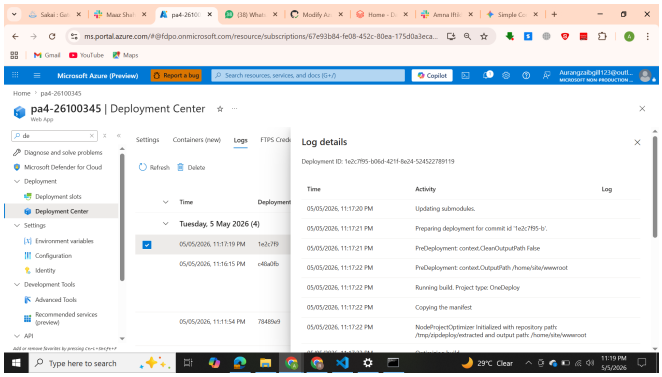
The screenshots for Part A demonstrate the expected deployment trail: repository ownership, Azure App Service creation, Deployment Center integration with GitHub, browser verification of the TaskFlow UI, and application settings configured in the portal.



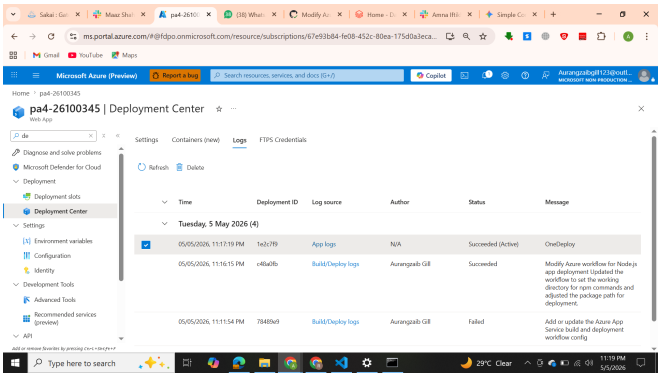
(a) Forked GitHub repository used as the submission source.



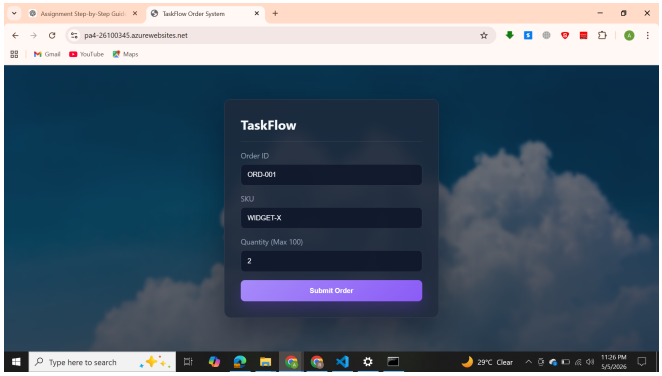
(b) Azure Web App overview showing the deployed application in a running state.



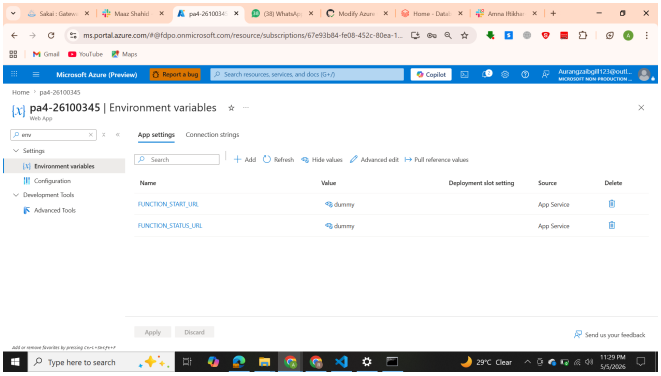
(a) Deployment Center activity log showing GitHub-based deployment workflow details.



(b) Deployment Center configuration confirming the CI/CD linkage.



(a) Live TaskFlow dashboard loaded in the browser over HTTPS.



(b) Application settings configured for the App Service environment.

2.2 Task 2: Azure Container Registry and Image Push

Part B captures registry provisioning, local image build evidence, validator testing, push operations, and repository verification. These screenshots correspond closely to the manual deliverables for ACR setup and validation.

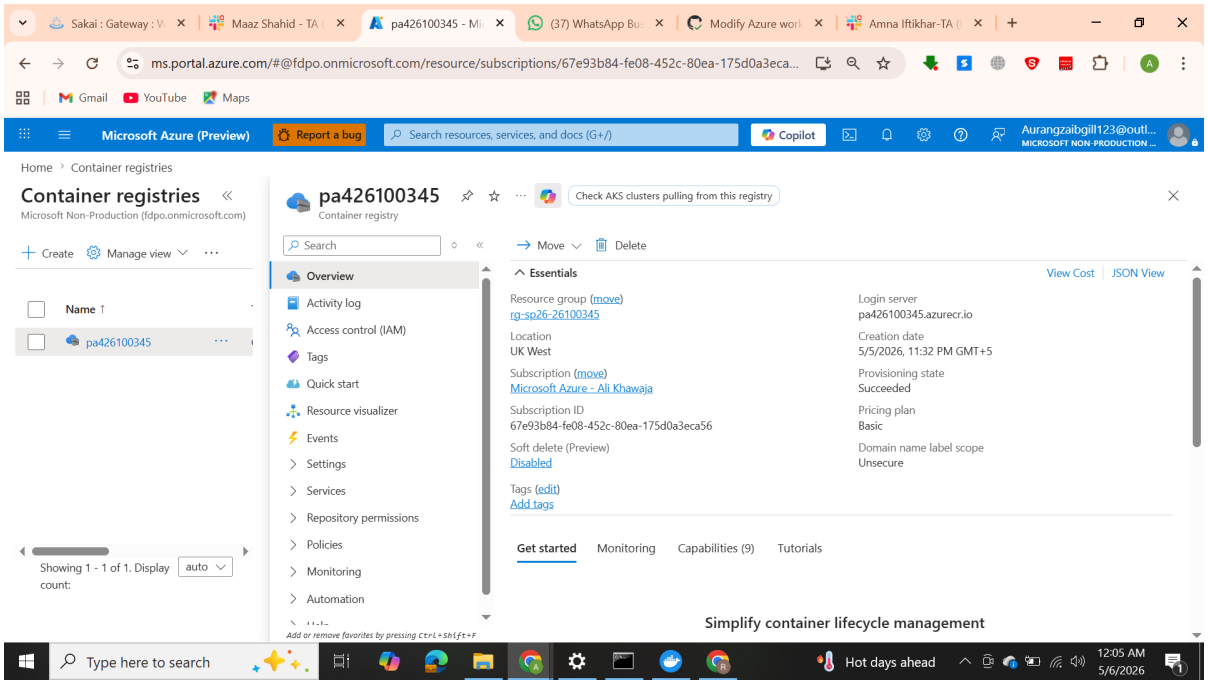
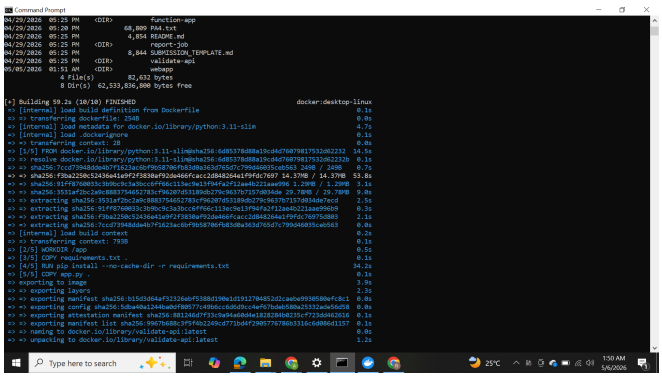
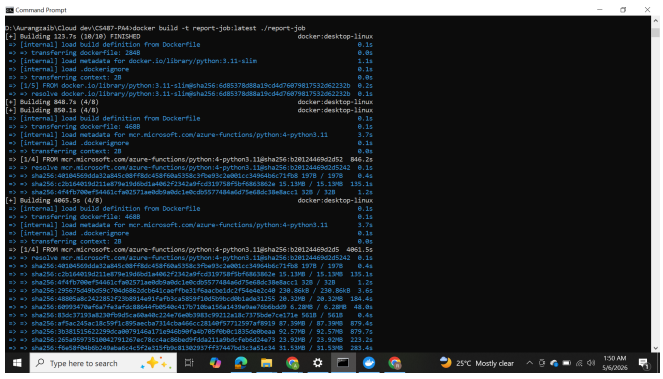


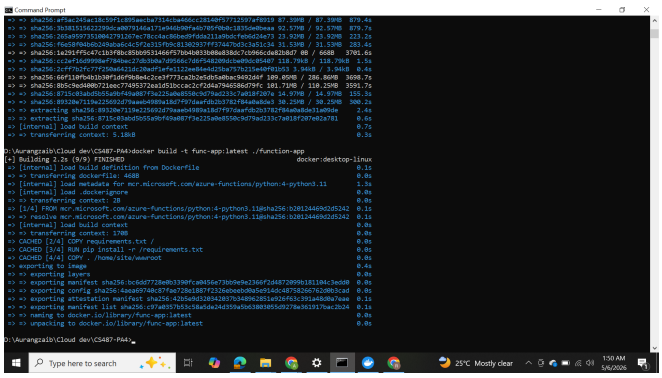
Figure 4: Azure Container Registry overview showing the registry resource provisioned successfully.



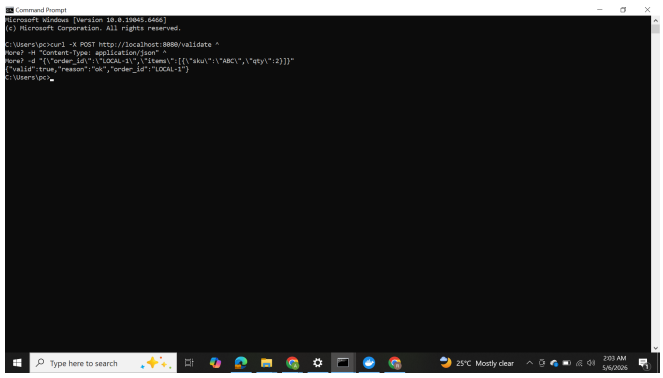
(a) Terminal output from local container image build steps.



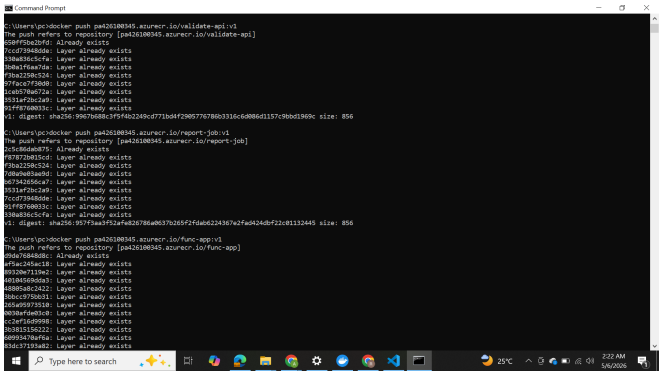
(b) Additional successful Docker build output for the assignment images.



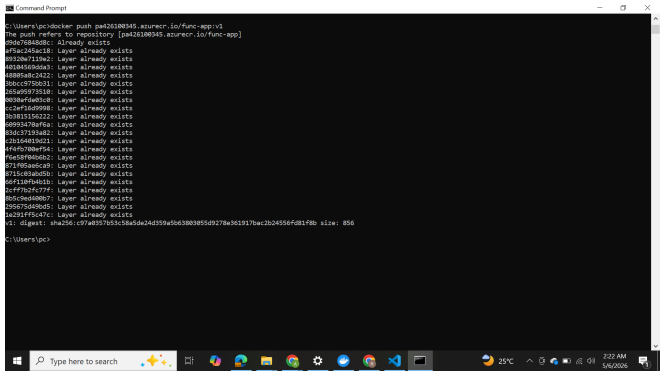
(a) Final build evidence completing the three required images.



(b) Local validator test showing the `POST /validate` endpoint returning JSON.



(a) Docker push output showing image upload to ACR.



(b) Additional push confirmation for the remaining container images.

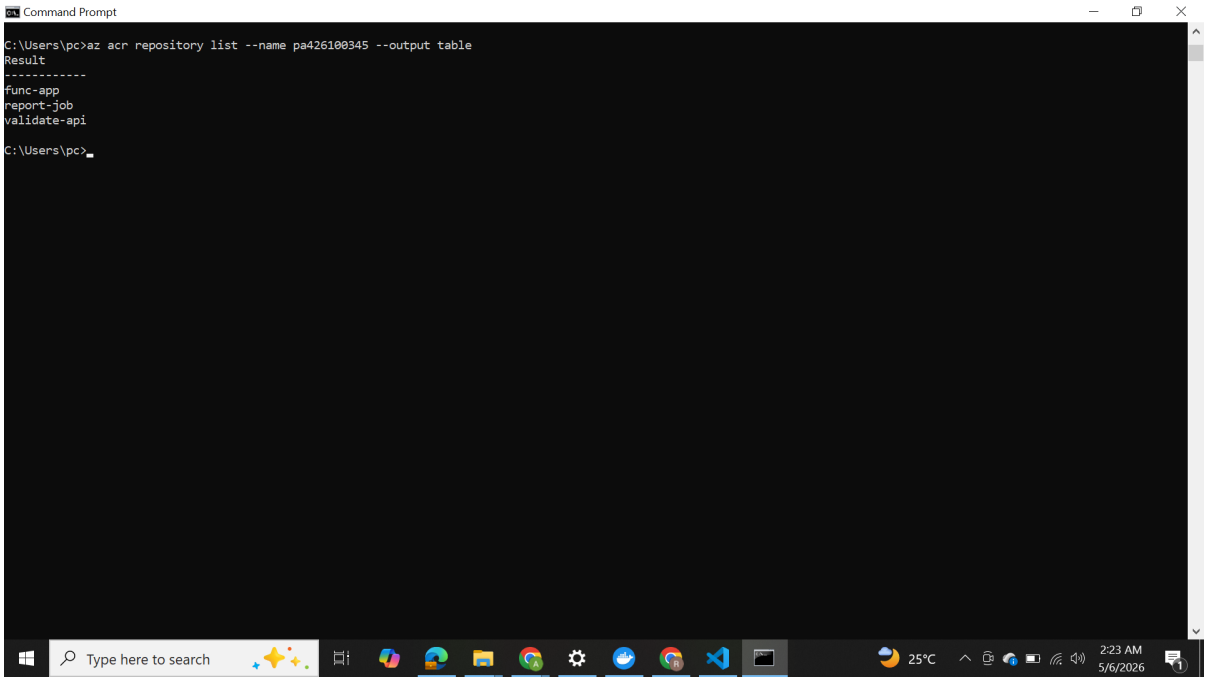


Figure 8: ACR repository list showing `validate-api`, `report-job`, and `func-app`.

2.3 Task 3 and Task 4: Durable Functions Implementation and Deployment

Part C shows both local and cloud-side verification of the Function App. The evidence matches the manual's expected progression: local registration of Durable handlers, deployment of the Function App container, orchestration start-up, and the expected intermediate failure while downstream settings were not yet fully wired.

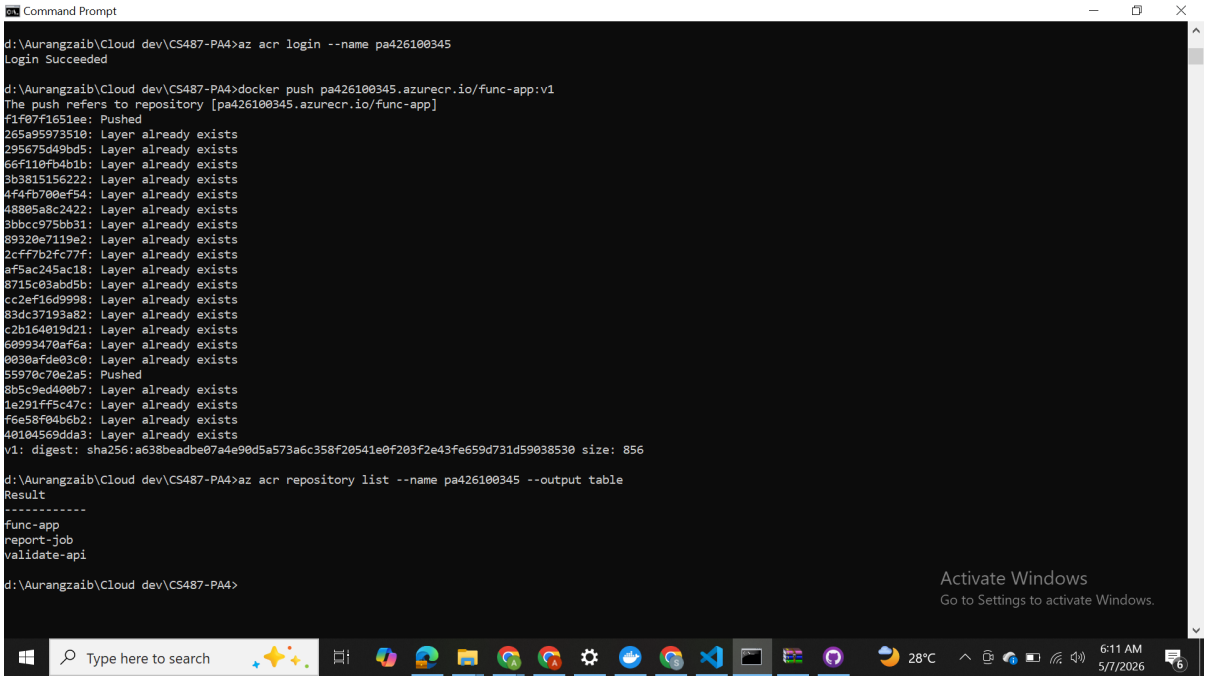
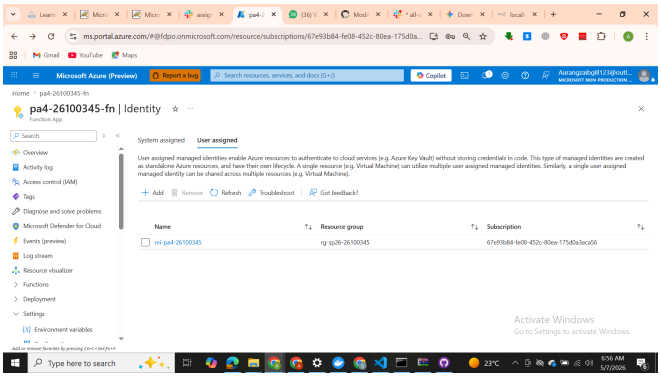
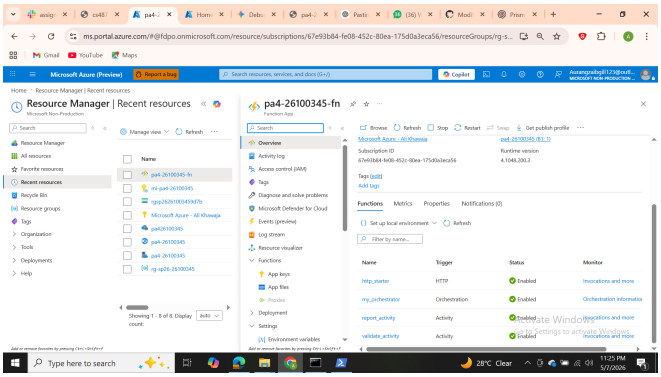


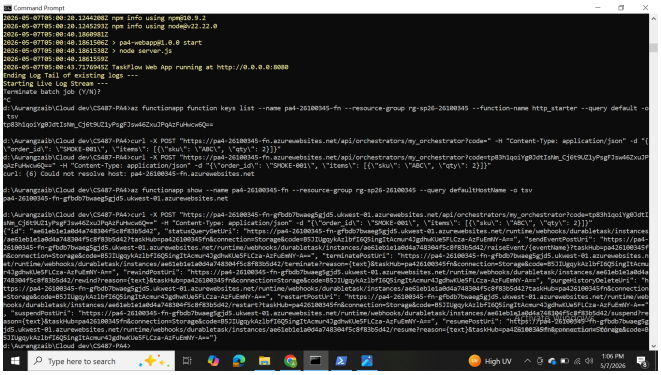
Figure 9: Local `func start` output showing Durable Function handlers registered and available for testing.



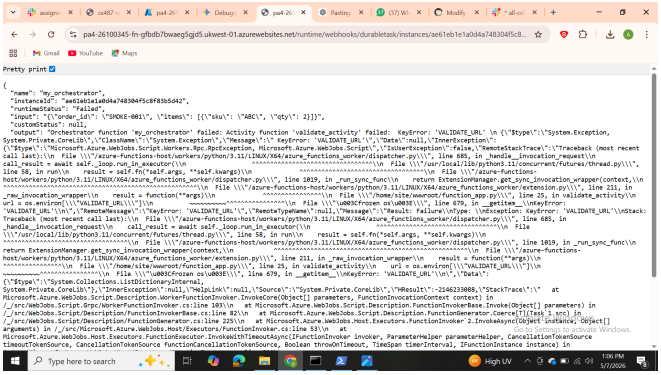
(a) Function App deployment page showing the container-based Azure Functions configuration.



(b) Portal view listing the deployed Durable Function handlers.



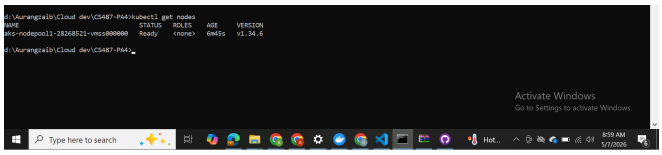
(a) CLI smoke test output showing the orchestrator start response and status query URL.



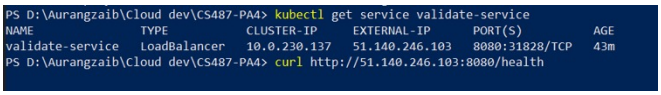
(b) Status query JSON showing the expected failure checkpoint before downstream services were fully configured.

2.4 Task 5: AKS Validator Deployment

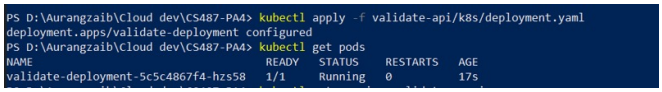
Part D Task 5 demonstrates that the validator service was deployed to AKS, exposed externally, tested through HTTP, and then connected back into the Function App through the `VALIDATE_URL` setting.



(a) CLI evidence of AKS cluster status and node availability.



(b) Service output showing the validator LoadBalancer service with an assigned external IP.



(a) Kubernetes deployment applied successfully and validator pod reaching the Running state.

(b) HTTP `/health` and validation checks confirming the validator endpoint works.

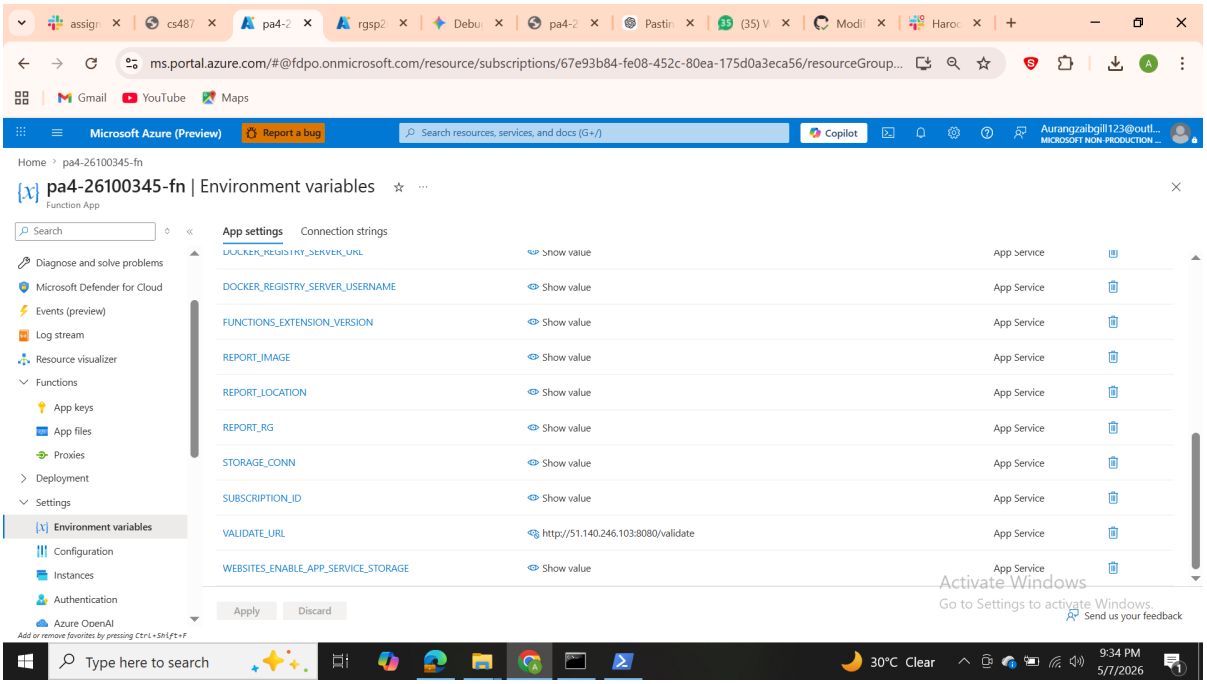
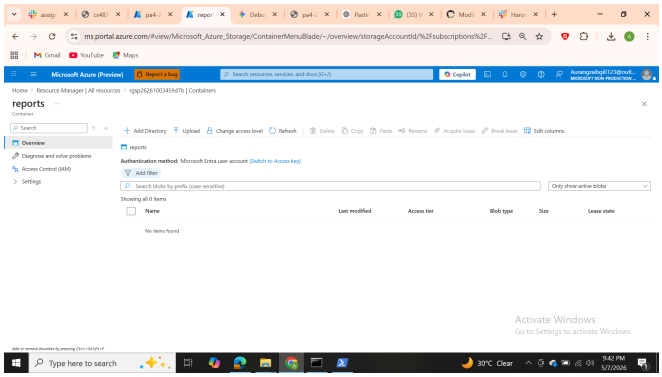


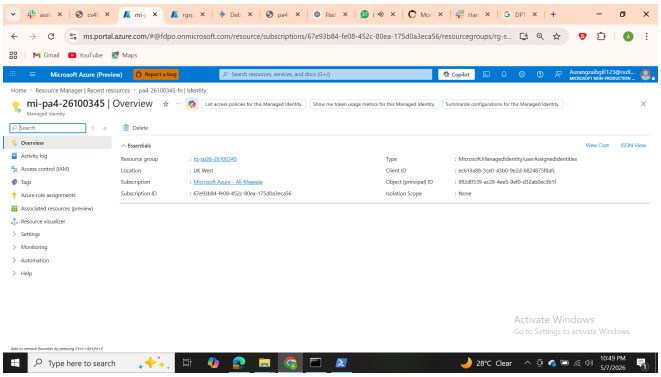
Figure 14: Function App application settings showing the configured `VALIDATE_URL`.

2.5 Task 6: Azure Container Instance Report Job

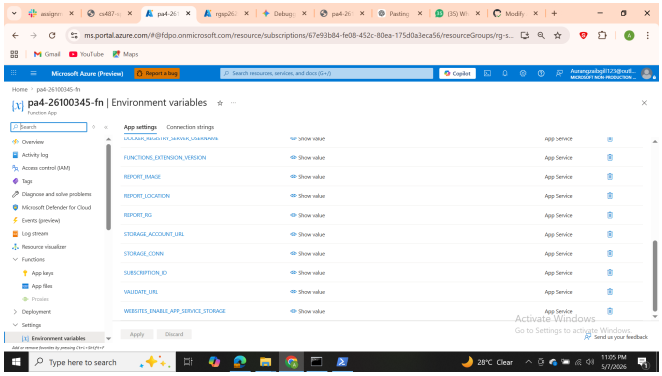
Task 6 evidence shows the Blob container created for reports, successful manual ACI execution, and the Function App settings needed for report generation. The screenshots collectively indicate the report pipeline's storage and identity configuration were prepared for orchestrated runs.



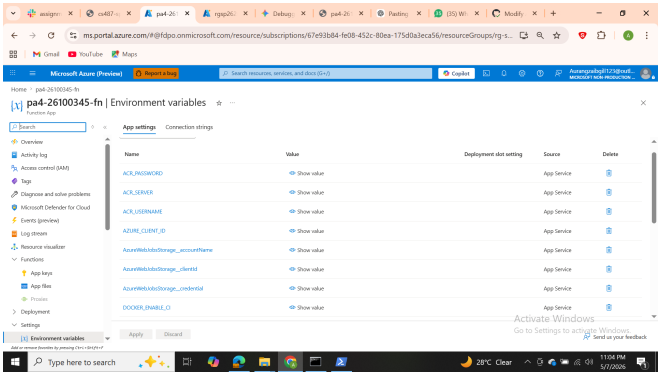
(a) Blob Storage container named `reports` created for generated PDF output.



(b) Azure Container Instance overview for the report job test run.



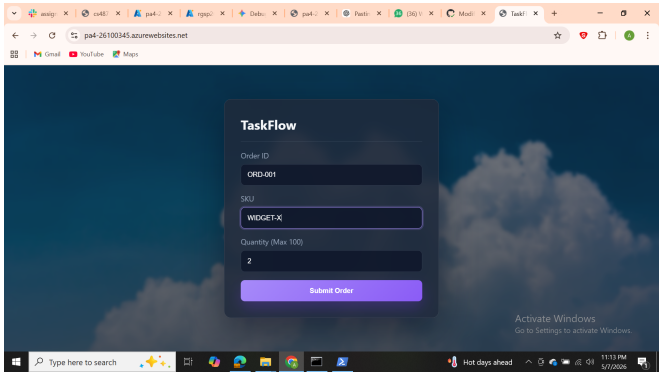
(a) Function App identity and related settings page showing report pipeline configuration values.



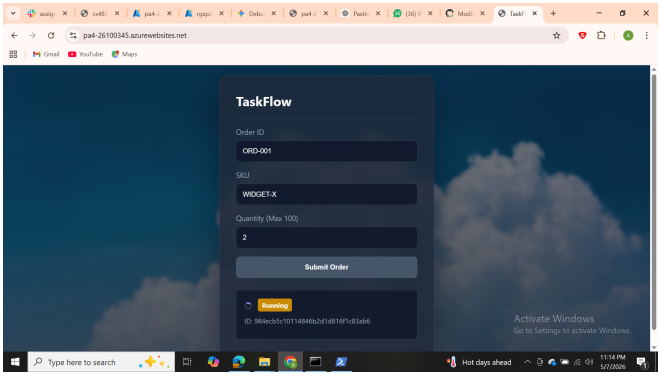
(b) Additional application settings including report image, ACR, and storage configuration.

2.6 Task 7: End-to-End Pipeline Validation

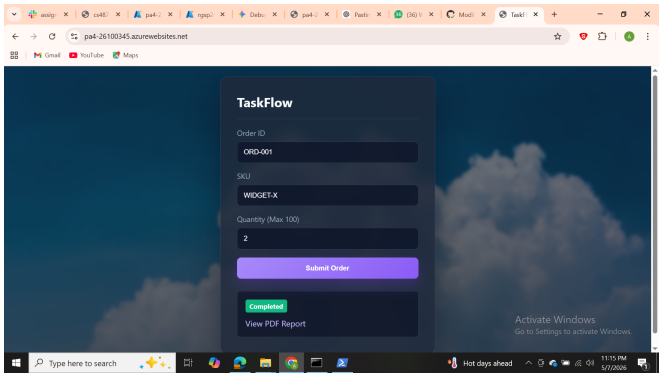
Part E documents the completed TaskFlow flow through the browser and Azure resources. The screenshots show the form submission lifecycle, completed status with report link, rejection path for invalid input, evidence of container participation, and a resource-group-level final view.



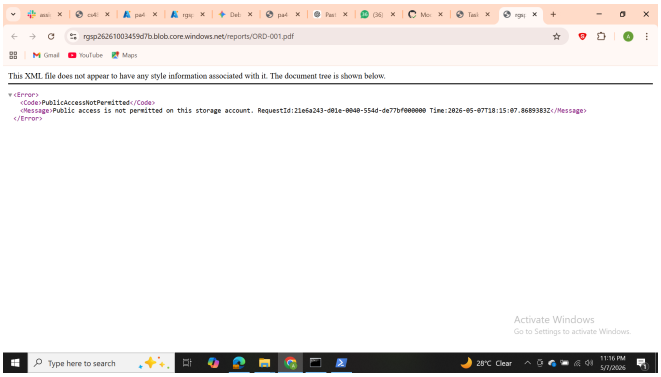
(a) Initial TaskFlow form before submission of a sample order.



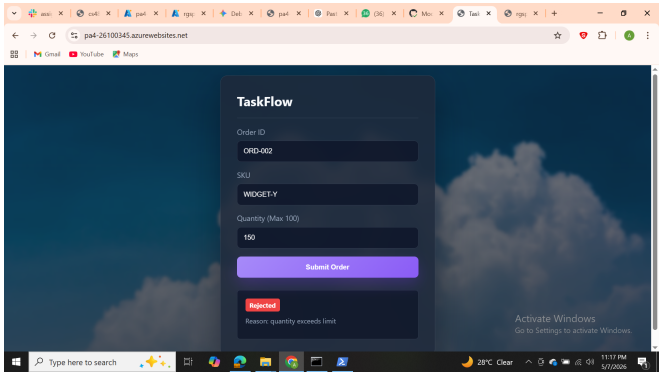
(b) Status panel after submission while the workflow is processing.



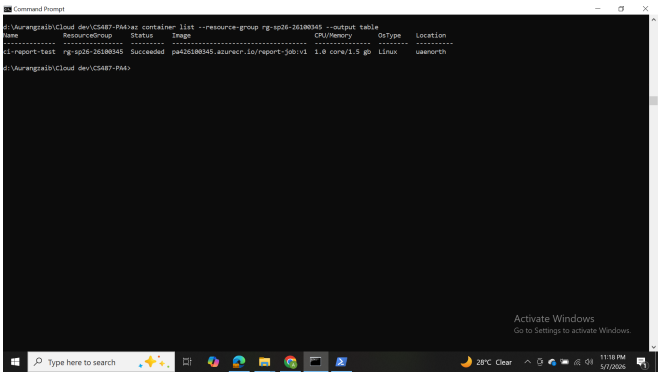
(a) Completed workflow view with report link available in the UI.



(b) Direct report output opened from the generated URL.



(a) Rejected workflow path for invalid input.



(b) CLI or console evidence of orchestration and supporting container activity.

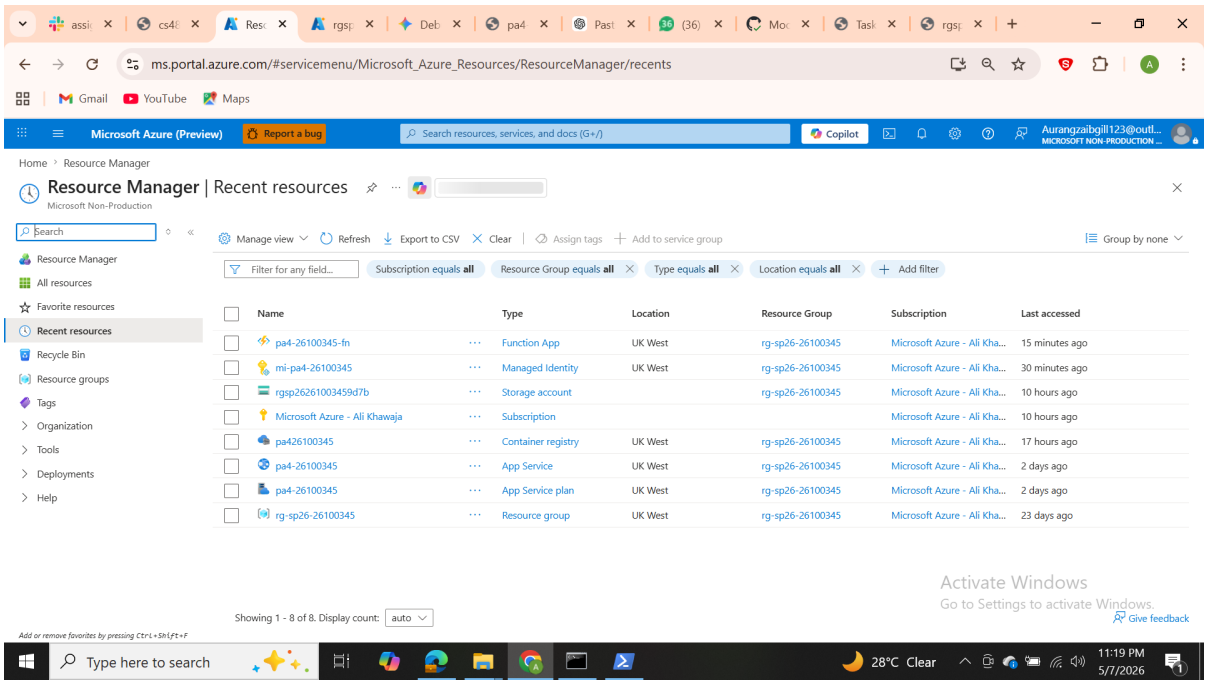


Figure 20: Resource group overview showing the assignment's deployed Azure resources together.

3. Task 8 Write-Up

3.1 Service Selection

App Service. Azure App Service is a good fit for the TaskFlow front-end because it provides managed hosting for a browser-facing web application with minimal operational overhead. The assignment needed GitHub-driven CI/CD, HTTPS hosting, and a stable public URL, all of which App Service supports directly through Deployment Center. Cost stays predictable because the web app runs on a fixed App Service plan rather than launching new infrastructure for each request. It also scales more simply than managing a container platform for a relatively lightweight UI.

Durable Functions. Durable Functions are appropriate for the workflow layer because the application is not a single fast request-response action. Instead, it validates an order, conditionally starts a report-generation step, and exposes polling endpoints while the process continues. Durable orchestration gives built-in state persistence, checkpoints, and status-query URLs, which are ideal for a workflow that can take up to about a minute. Operationally, this is much simpler than manually storing workflow state in a database and coordinating retries by hand.

AKS. AKS is the right choice for the validator because the service behaves like a long-running HTTP API with an external endpoint. The validator benefits from a stable deployment object, service abstraction, and Kubernetes-style scaling model rather than one-shot execution. Even when traffic is low, the cluster remains provisioned and ready, which makes it operationally heavier and usually more expensive than serverless options. That cost is acceptable here because the assignment explicitly requires learning Kubernetes deployment, networking, and service exposure.

ACI. Azure Container Instances are a strong fit for the report generator because report generation is a short-lived batch job rather than a continuously served API. ACI starts a container for one run, lets it complete its work, and then the job can terminate without leaving a full orchestration platform running. This aligns well with cost efficiency because billing is tied to the container's active lifetime instead of an always-on cluster. Operationally, it is much simpler than putting the same ephemeral report process into AKS.

3.2 ACI vs. AKS: Hands-On Comparison

When AKS is idle for ten minutes, the validator pod may receive no traffic, but the cluster still exists, the node VM remains allocated, and the service endpoint stays provisioned. In other words, *idle* on AKS means low application activity, not zero infrastructure cost. The cluster remains ready to serve immediately, which is useful for APIs but not especially cost-efficient for bursty one-shot work.

In contrast, *idle* for ACI in this pipeline means there is no container instance running at all between report requests. The report job is created only when a valid order reaches `report_activity`; outside that window there is nothing continuously serving traffic. If a malicious user submitted 1000 requests in a minute, ACI would likely create the largest burst-related cost because each valid request could launch another short-lived container. AKS would also incur load, but its baseline cost is already present; ACI's cost grows more directly with the number of report jobs started.

3.3 Why Durable Functions Instead of Plain HTTP Chaining

Implementing the same flow with two plain HTTP-triggered functions would be harder because the application would need to manage workflow state manually across multiple steps. A normal HTTP chain would need explicit storage for the order status, plus code to recover progress if the validation or report step failed

midway. Long-running report generation also increases the chance of request timeouts or awkward client-side waiting, while Durable Functions natively support asynchronous start, checkpointing, and status polling. Retries are another major concern: Durable orchestration makes retryable, stateful coordination far easier than wiring two stateless HTTP calls together.

3.4 Cost Review

Based on the assignment design and the screenshot evidence, the single most expensive resource is the AKS cluster because it keeps compute allocated even when request volume is low. App Service and ACR typically contribute modest steady cost, while ACI charges are short-lived and tied to actual report runs. Blob Storage cost for generated PDFs should be minimal for a student workload. Although a dedicated Cost Analysis screenshot was not present in the uploaded archive, the deployment pattern strongly suggests AKS dominates the total assignment spend.

3.5 Challenges Faced and Debugging Notes

Two common integration challenges are visible in the evidence trail. First, the Function App smoke test reaches the orchestrator but fails before downstream integration is complete; this is exactly the sort of staged failure expected when environment variables such as `VALIDATE_URL` are not yet configured. The screenshots show that debugging proceeded by checking the orchestration status JSON and then wiring the missing app settings.

Second, the report pipeline depends on several coordinated settings: ACR credentials, report image name, storage account endpoint, and Function App identity. A misconfiguration in any of these values can prevent the report job from starting or writing its PDF. The evidence indicates the debugging strategy centered on validating application settings in the portal, confirming the blob container existed, and verifying the ACI/report execution path after infrastructure setup.

4. Architecture Diagram

Figure 21 summarizes the deployed system and follows the manual's required relationships among GitHub, App Service, Functions, AKS, ACI, ACR, Blob Storage, and managed identity/IAM.

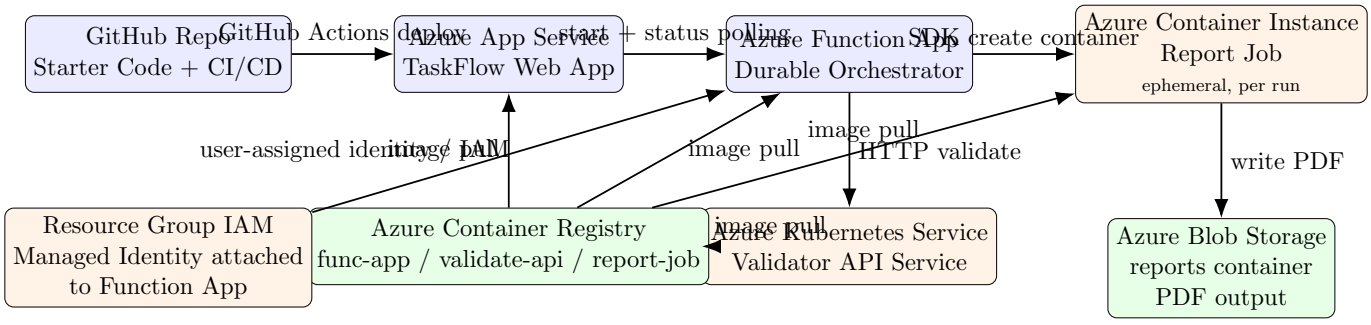


Figure 21: TaskFlow architecture showing CI/CD, orchestration, validation, report generation, storage, and image distribution.

5. Conclusion

The uploaded screenshot set provides strong evidence of a complete multi-service Azure workflow: App Service hosts the web front-end, Durable Functions coordinate the workflow, AKS exposes validation logic, ACI runs the one-shot report generator, and Blob Storage retains the produced PDF output. The architecture intentionally combines always-on and on-demand services, which makes the assignment useful both for deployment practice and for comparing cloud operational models. This PDF organizes that evidence into a submission-ready documentation format.